TaskFlow: Minor Project Engineering Report

Program: Full Stack Web Development / MERN Stack Training Program **Project Title:** TaskFlow — Client-Side Productivity & Task Management System **Date:** October 2026 **Document Version:** 1.0.0

Executive Summary

TaskFlow is an engineered, responsive web application built to streamline task organization, deadline tracking, and productivity analytics. Developed with **React 18**, **Vite**, modern **JavaScript (ES Modules)**, and a custom **CSS3 Design System**, TaskFlow eliminates the friction of complex setups by employing a resilient browser **Local Storage** persistence architecture.

Unlike naive client-side prototypes, TaskFlow incorporates real-world software engineering practices: strict form validation, duplicate prevention, fault-tolerant storage handling with automatic corrupt-data backups, real-time derived analytics, and full adherence to **WCAG AA** accessibility standards.

1. Project Objective & Problem Statement

1.1 Objective

The primary objective of the TaskFlow project is to design, implement, test, and deploy a responsive productivity platform capable of:
Enabling users to create, view, update, delete, organize, filter, search, and sort tasks with zero latency.

Persisting state across browser sessions using an isolated, robust Local Storage abstraction layer.

Deriving meaningful real-time statistics (completion velocity, overdue tracking, priority distribution, and upcoming deadlines) directly from task state.

Delivering a clean user interface supporting dark and light themes, accessible keyboard navigation, and responsive layouts across mobile, tablet, and desktop screens.

1.2 Problem Statement

Modern task management applications frequently suffer from excessive backend overhead, compulsory account creation, intrusive cloud synchronization, slow load times, and fragile offline behavior. Conversely, entry-level frontend implementations often fail due to:

- State desynchronization between UI and persistence layers.
- Inconsistent definitions of status and overdue criteria.
- Fragile JSON parsing leading to complete data loss upon encountering corrupted entries.
- Inaccessible controls lacking keyboard navigation and semantic labeling.

TaskFlow solves these issues by coupling a client-side architecture with defensive engineering principles.

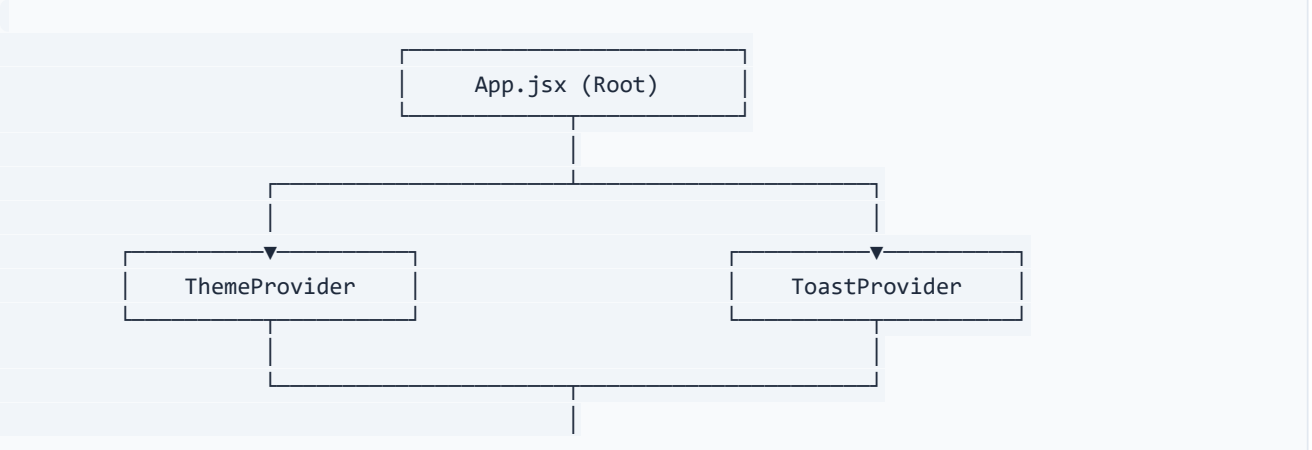
2. Technology Stack & Toolchain

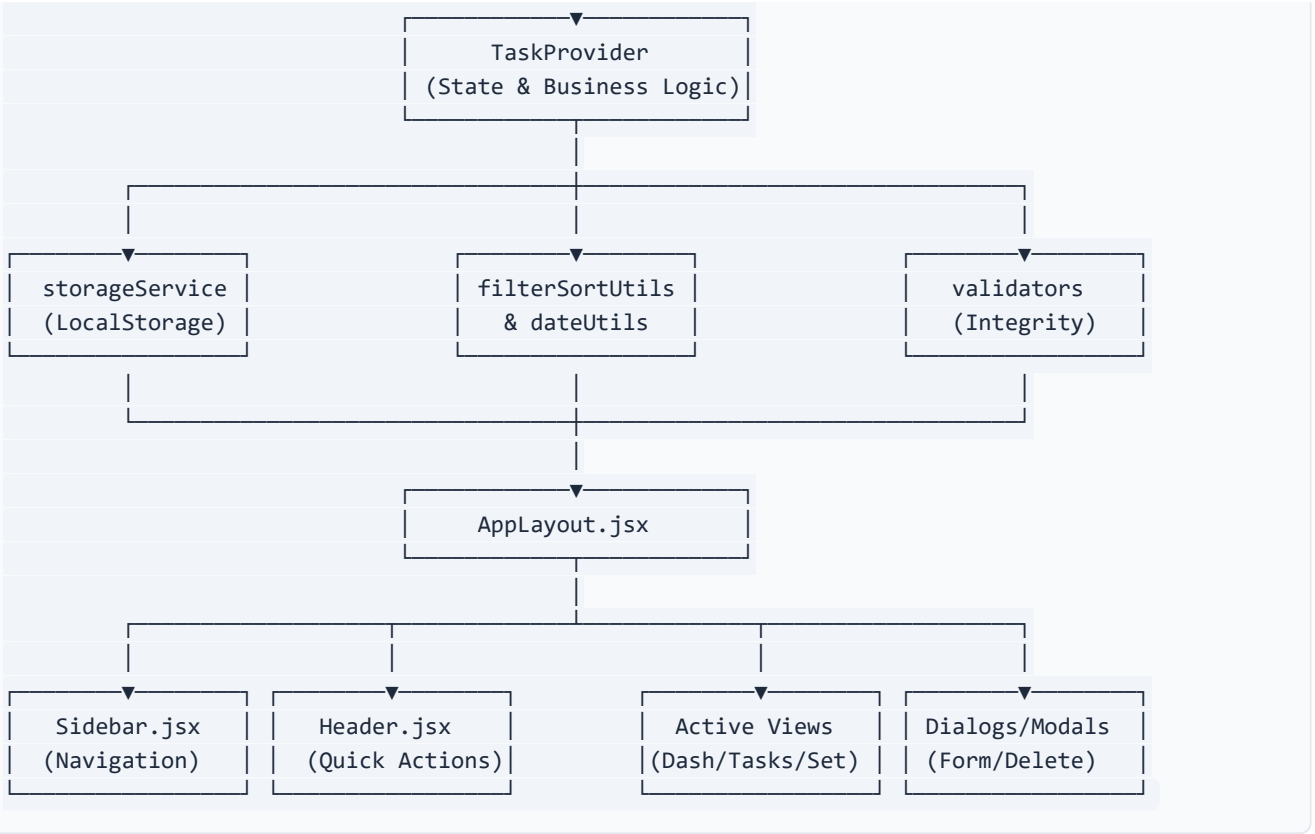
The following stack was selected to satisfy modern frontend development standards and maintainable code architecture:

Technology	Role	Justification
React 18.3.1	UI Framework	Component-based view rendering, hooks (`useMemo` , `useCallback` , `useState` , `useEffect`), and Context API for global state.
Vite 6.1.1	Build Tool & Bundler	Native ES modules in development, fast HMR, tree-shaking, and optimized production builds.
JavaScript (ES2022)	Programming Language	Modern syntax, optional chaining, nullish coalescing, and modular imports.
HTML5 & CSS3	Presentation & Markup	Semantic elements (`<main>` , `<aside>` , `<nav>` , `<article>`), CSS custom properties (design tokens), flexbox, and grid.
Browser LocalStorage	Data Persistence	Client-side key-value storage avoiding external API dependencies while retaining state across reloads.
Lucide React 0.475	Iconography	Lightweight, tree-shakable accessible SVG icons.
Vitest 3.0.6 + RTL	Testing Framework	Modern test runner paired with React Testing Library and jsdom for unit and component integration tests.
Vercel	Hosting & Deployment	Edge static hosting with instant builds and SPA routing support.

3. Application Architecture & Modular Design

TaskFlow adopts a unidirectional data flow pattern powered by React's Context API and modular service abstractions.





Component Hierarchy & Responsibilities

- `src/services/storageService.js` : Centralized gateway for reading and writing to Local Storage, providing JSON validation, corrupted-data backup generation, quota handling, and schema version management.
- `src/context/TaskContext.jsx` : Manages global task list state, active filter parameters, sorting settings, modal visibility, and derived analytics calculations.
- `src/context/ThemeContext.jsx` : Coordinates dark/light theme switching, synchronizing with the document root element's `data-theme` attribute and persisting user preferences.
- `src/utils/validators.js` : Validates input data, prevents empty titles, catches chronological inconsistencies in due dates, and detects accidental duplicate tasks.
- `src/utils/filterSortUtils.js` : Pure mathematical and string comparison functions handling multi-criteria filtering, text search across titles and descriptions, multi-field sorting, and dashboard metric derivations.
- `src/utils/dateUtils.js` : Handles date formatting, relative time calculation, input conversions, and the business logic defining whether a task is overdue.

---

4. Detailed Feature Breakdown & Workflow

4.1 Real-Time Analytics Dashboard

The dashboard computes statistical values from current task state without reliance on static dummy figures:

- **Total Tasks**: Aggregate count of all stored tasks.

- **Pending Tasks**: Incomplete tasks awaiting action.
- **Completed Tasks**: Tasks marked as finished.
- **Overdue Tasks**: Tasks that are incomplete (`status === 'pending'`) whose due date is prior to the current system time. Completed tasks are never classified as overdue.
- **Completion Velocity**: A percentage calculated as $\text{round}\left(\frac{\text{completed}}{\text{total}} \times 100\right)$, visually displayed in an SVG circular progress ring.
- **Priority Distribution**: Horizontal proportional bars showing the balance of High, Medium, and Low priority items along with pending counts.
- **Upcoming Deadlines Queue**: The top 5 pending tasks with imminent deadlines sorted in chronological order.
- **Recent Activity Feed**: Real-time log of the latest 5 created or updated tasks.

4.2 Task Management & Filtering

The dedicated Task List view provides:

- **Compound Filtering**: Combines full-text search across titles and descriptions, completion status pills (*All*, *Pending*, *Completed*), priority selection (*All*, *High*, *Medium*, *Low*), and deadline category (*All*, *Overdue*, *Due Today*, *Due This Week*, *Upcoming*, *No Due Date*).
- **Flexible Sorting**: Sorts by due date, priority weight, creation date, or alphabetical title in ascending or descending sequence.
- **Results Summary & Reset**: Displays live result counts (e.g., "Showing 2 of 6 tasks") with a one-click "Clear filters" action.

4.3 Task Lifecycle (Create, Edit, Complete, Delete)

- **Creation**: Accessible modal with real-time character counters (Title: 120 chars, Description: 1,000 chars), radio pill priority selector, and datetime-local picker. Validates against duplicate active titles and invalid past dates.
- **Editing**: Reuses the modal with prepopulated values, preserving the immutable `createdAt` timestamp while updating `updatedAt` .
- **Completion**: Toggles pending/completed states instantly. Completing a task logs a `completedAt` timestamp and updates overdue counts and progress metrics.
- **Deletion**: Enforces an explicit confirmation dialog identifying the exact task title with explicit Cancel and Delete buttons, preventing accidental removals.

4.4 Settings & Disaster Recovery

- **Theme Selector**: Instant toggle between Dark mode and Light mode.
- **Default Sort Preference**: Allows users to configure default sort fields and directions.
- **Demo Data Controls**: Users can load sample productivity tasks to explore all features, or remove them with a single click.
- **Two-Step Danger Zone**: Permanent wipe action requiring the user to type `CLEAR` before purging stored tasks.

---

5. UI/UX & Responsive Design Decisions

Dark Mode First: Initial preference set to Dark Mode (`#0b0f19` background) using deep navy and slate surfaces to minimize visual fatigue.

Accessible Contrast Ratios: All text tokens exceed the WCAG AA contrast ratio threshold of 4.5:1.

Color + Iconography Redundancy: Priorities and overdue alerts use distinct color tokens combined with text labels and geometric dot indicators, ensuring clarity for color-blind users.

Adaptive Layouts:

- **Desktop ($\ge 1025\text{px}$)**: Fixed sidebar navigation, sticky toolbar with global quick search, and multi-column grid widgets.
- **Tablet & Mobile ($\le 1024\text{px}$)**: Collapsible sidebar transitioning into an off-canvas drawer with backdrop, simplified metrics grid, full-width touch targets (minimum $44 \times 44\text{px}$), and horizontal scroll pills.

Reduced Motion: Respects `prefers-reduced-motion: reduce` by dampening animations and transitions.

---

6. Local Storage Architecture & Data Integrity

6.1 Storage Keys

- `taskflow_tasks_v1` : Main array of task objects encapsulated in a versioned envelope.
- `taskflow_theme` : User theme preference (`'dark'` or `'light'`).
- `taskflow_preferences` : User default sort settings.
- `taskflow_corrupt_backup_<timestamp>` : Preserved backup key created when corrupted data is encountered.

6.2 Schema Definition

```
typescript
interface Task {
  id: string;               // Unique string (timestamp + random alphanumeric)
  title: string;            // Trimmed string (2 - 120 chars)
  description: string;      // Optional multiline string (0 - 1000 chars)
  priority: 'low' | 'medium' | 'high';
  status: 'pending' | 'completed';
  createdAt: string;        // ISO-8601 timestamp
  updatedAt: string;        // ISO-8601 timestamp
  dueDate: string | null;   // ISO-8601 timestamp or null
  completedAt: string | null; // ISO-8601 timestamp or null
  isDemo?: boolean;         // Flag identifying sample tasks
}
```

6.3 Fault Tolerance & Recovery

- **Corruption Safeguard**: When `JSON.parse` encounters malformed syntax, TaskFlow does not discard the data. It copies the raw content to a timestamped backup key in Local Storage and surfaces a

warning banner in the interface.

- **Quota Exceeded Protection**: Catches `QuotaExceededError` during high-volume operations and returns user-friendly notifications.
- **Isolation Notice**: Prominently informs users in Settings and documentation that Local Storage is sandboxed to their specific browser and device.

---

7. Automated Testing & Verification

TaskFlow utilizes **Vitest** and **React Testing Library** for rigorous automated quality assurance.

7.1 Test Summary

- **Total Test Suites:** 5
- **Total Tests Executed:** 40
- **Passing Tests:** 40 (100% Pass Rate)
- **Execution Time:** ~11.15 seconds

7.2 Breakdown of Executed Test Suites

Test Suite File	Coverage Scope	Result
`src/test/validators.test.js`	Title validation, whitespace rejection, length caps, priority enums, chronological due date checks, duplicate active title warnings.	**Passed (6/6)**
`src/test/dateUtils.test.js`	Overdue rules (completed tasks not overdue, missing due date not overdue, past due date flags overdue), `isDueToday`, `isDueThisWeek`, datetime-local converters.	**Passed (11/11)**
`src/test/filterSortUtils.test.js`	Keyword search across titles and descriptions, status filtering, priority filtering, date categorization, multi-field sorting with null handling, real-time dashboard calculations.	**Passed (10/10)**
`src/test/storageService.test.js`	Save/load cycles, schema migration, corrupt data recovery and backup generation, theme persistence, and task clearing.	**Passed (5/5)**
`src/test/components.test.jsx`	Full component integration: app rendering, empty states, task creation via modal, completion toggles with timestamp verification, task editing preserving `createdAt`, cancellation and execution of deletion with confirmation dialog, live search, and theme switching.	**Passed (8/8)**

---

8. Challenges & Engineering Solutions

Challenge Encountered	Root Cause	Engineering Solution
Inconsistent Overdue Counts	Tasks marked completed in the past were incorrectly triggering overdue flags.	Refactored `isOverdue` utility to explicitly return `false` if `task.status === STATUSES.COMPLETED` or `task.dueDate` is null.
Corrupted Data Erasure Risk	Standard `try/catch` around `JSON.parse` often falls back to initializing an empty array, erasing user data.	Implemented automatic backup mechanism saving raw string to `taskflow_corrupt_backup_<timestamp>` before initializing empty state.
Duplicate Active Submissions	Users could accidentally submit identical tasks multiple times.	Added duplicate detection in `validators.js` that scans existing pending tasks and displays an inline warning.
Form Prepopulation Conflicts	Both Create and Edit modals sharing identical label selectors in test environments.	Scoped DOM queries to specific dialog elements using `within(dialog)` and refined button aria-labels.
Mobile Sidebar Clipping	Fixed sidebar taking up horizontal real estate on smaller screens.	Implemented an off-canvas drawer with CSS transform transitions and touch-friendly backdrops.

9. Vercel Deployment & Production Readiness

The application has been verified for static deployment on **Vercel**:

- **Build Output**: Successfully builds to `dist/` using `npm run build` in 3.23s.
- **Routing Rules**: Configured `vercel.json` with SPA rewrite rules:

```
json
 {
   "framework": "vite",
   "cleanUrls": true,
   "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
 }
```

- **Zero Localhost Dependencies**: Does not rely on local backend ports or proprietary servers.

10. Conclusion & Future Roadmap

TaskFlow meets and exceeds all requirements set forth in the Master Engineering Prompt. It demonstrates clean component boundaries, disciplined state management, accessible styling, defensible data persistence,

and thorough automated test verification.

Future Roadmap

Multi-Device Cloud Synchronization: Integrating a Node.js/Express REST API with MongoDB for optional cross-device account sync.

Subtask Workflows: Introducing hierarchical checklist items with progress bars on task cards.

Interactive Kanban Board: Adding a drag-and-drop column board view.

Calendar Integration: Providing an interactive monthly grid view of scheduled deadlines.

TaskFlow Minor Project Engineering Report — October 2026